Browser interface and hosting

Run locally

Use Python 3.11 or newer. Download the interface branch from GitHub using **Code → Download ZIP**, extract it, and open the extracted folder.

- **Windows:** double-click `start-interface.bat`.
- **macOS/Linux:** open a terminal in the folder and run `sh start-interface.sh`.

The launcher creates a virtual environment, installs dependencies, and opens the interface at http://localhost:8501. Installation needs Internet access. Keep the terminal open while presenting and press Ctrl+C when finished. Show the browser window or share your screen with colleagues; the scraper runs on your computer. A localhost link works only on that computer.

Alternatively, run these commands from the repository directory:

```
python -m pip install -r requirements.txt
python -m streamlit run streamlit_app.py
```

The initial mode runs the existing synthetic, offline demonstration. It does not search real locations: the selected locations and form values apply to live research. Demo downloads contain invented policies, not research findings. Demo mode hides the live form because it uses fixed examples. All application labels and the default codebook are Swedish; technical Excel sheet names and log events remain unchanged. See LAS_MIG.pdf for Swedish launch and usage instructions.

For live runs, choose Kommuner, Regioner, or Både kommuner och regioner (a mixed list), select one or more authorities, or use Alla i listan / Rensa val, and enter the question, keywords and query templates. Choose OpenAI or local Ollama for classification. The local option defaults discovery to crawl, needs no OpenAI key, and applies the selected model to classification, verification and adjudication. The connection test lists installed models without running inference. See docs/lokal-ai.pdf. OpenAI discovery methods still require an OpenAI key, even when classification uses Ollama. Crawl + collect only needs no key. The Swedish interface includes all 290 municipalities. Their names, official website hosts and codes are matched against SKR; see examples/registry_sources/README.pdf. The six-city pilot list is retained as examples/municipalities_pilot.csv. The region list has all 21 regional authorities, including Gotland. These are authority websites, not searches of every municipality inside the selected region. Review domains and add external document archives to your CSV as needed; this change has not checked website availability.

Upload a replacement CSV with `id,name,domains,seeds,kind`; the final two columns are optional. `kind` is `kommun` (the default for legacy CSVs) or `region`. Separate multiple domains/seeds with semicolons. IDs must be unique across both types. Region IDs use a `region-` prefix to prevent collisions with municipality codes.

The default codebook asks about language requirements for the authority's staff. Changing keywords affects discovery ranking; changing the question does not rewrite classification labels or instructions. Upload a YAML codebook to research a different topic or use the historical elderly-care codebook with its required cutoff. Regions and municipalities have different

responsibilities: the app does not assume regions run municipal elderly care. Queries accept `{location}`, `{domain}`, `{question}`, and the legacy `{municipality}` alias. `{region}` is populated only for region records.

Existing CLI commands remain supported. Regions also work without the browser:

```
municipal-research run --config examples/interface.yaml --locations examples/regions.csv --run-
dir runs/regions --limit 1
```

Saved outputs retain legacy `municipality`/`municipality_id` field names for compatibility, with `location_type` added to Summary and full type information in `municipalities.json`. Separate run directories and per-session caches prevent visitors from writing to the same run/cache. The UI polls job status and recent audit events; it does not estimate percent completion. Stop terminates the child process and may leave an incomplete manifest. A new run uses a new directory.

Share with colleagues on Streamlit Community Cloud

1. Sign in at https://share.streamlit.io/ with your GitHub account.
2. Create an app from `Mihagley/Scraping_Swedish_elderly_care`.
3. Select the branch containing this interface and set the entrypoint to `streamlit_app.py`. Choose Python 3.11 or newer in advanced settings.
4. Deploy and share the resulting app URL with colleagues.

The repository-root `requirements.txt` installs the app and demo dependencies. You can deploy the feature branch before merging. No account-level API secret is needed: visitors enter their own keys. Use the offline demo for a no-cost presentation. Research may use paid API calls; the call limit is not a spend cap.

Cloud files are temporary and disappear on app restart; download results after each run. Jobs are tied to the running app instance. This is a small-team demonstration setup; long-running shared research needs durable storage and a job worker. Hosting access controls are configured in your Streamlit account.